Step 3 — Deciding What's Worth Remembering, and Writing It Back — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and saying no
- Hint 2 — The plan, and the stable key

Hint 1 — The idea, and saying no

A `write_back_node` (after `chat`) decides whether the user's last message holds a durable fact. The real skill is saying **no** most of the time. Add:

- `src/memory/writer.py` — a `MemoryDecision` Pydantic model (`should_remember`, `key`, `fact`) and `decide_memory(model, message)` = `prompt | model.with_structured_output(MemoryDecision)`.
- `src/prompts/writer.txt` — the prompt that decides: durable facts (name, lasting preference, allergy) yes; ephemera (today's dinner, one-off questions) no; reuse a stable key.
- `write_back_node` in `nodes.py` — find the last user message, `decide_memory`, and `save_fact` only if `should_remember`.

Hint 2 — The plan, and the stable key

```
writer.py:  MemoryDecision(should_remember, key, fact)
            decide_memory = prompt | model.with_structured_output(...)
nodes.py:   make_write_back_node(model) -> write_back_node(state, *, store):
    last = last HumanMessage in state["messages"]
    d = decide_memory(model, last.content)
    if d.should_remember:
        save_fact(store, state["user_id"], d.key, d.fact)
graph.py:   START -> recall -> chat -> write_back -> END
```

Two things: (1) the node needs the model **and** the store — capture the model in a closure, and declare `store` for injection. (2) the prompt must insist on a **stable key** (`preferred_name` for the

name), so "call me Sammy" updates the same fact instead of adding a second. Test the ratio: most messages save nothing. The whole file set is in the Solution.